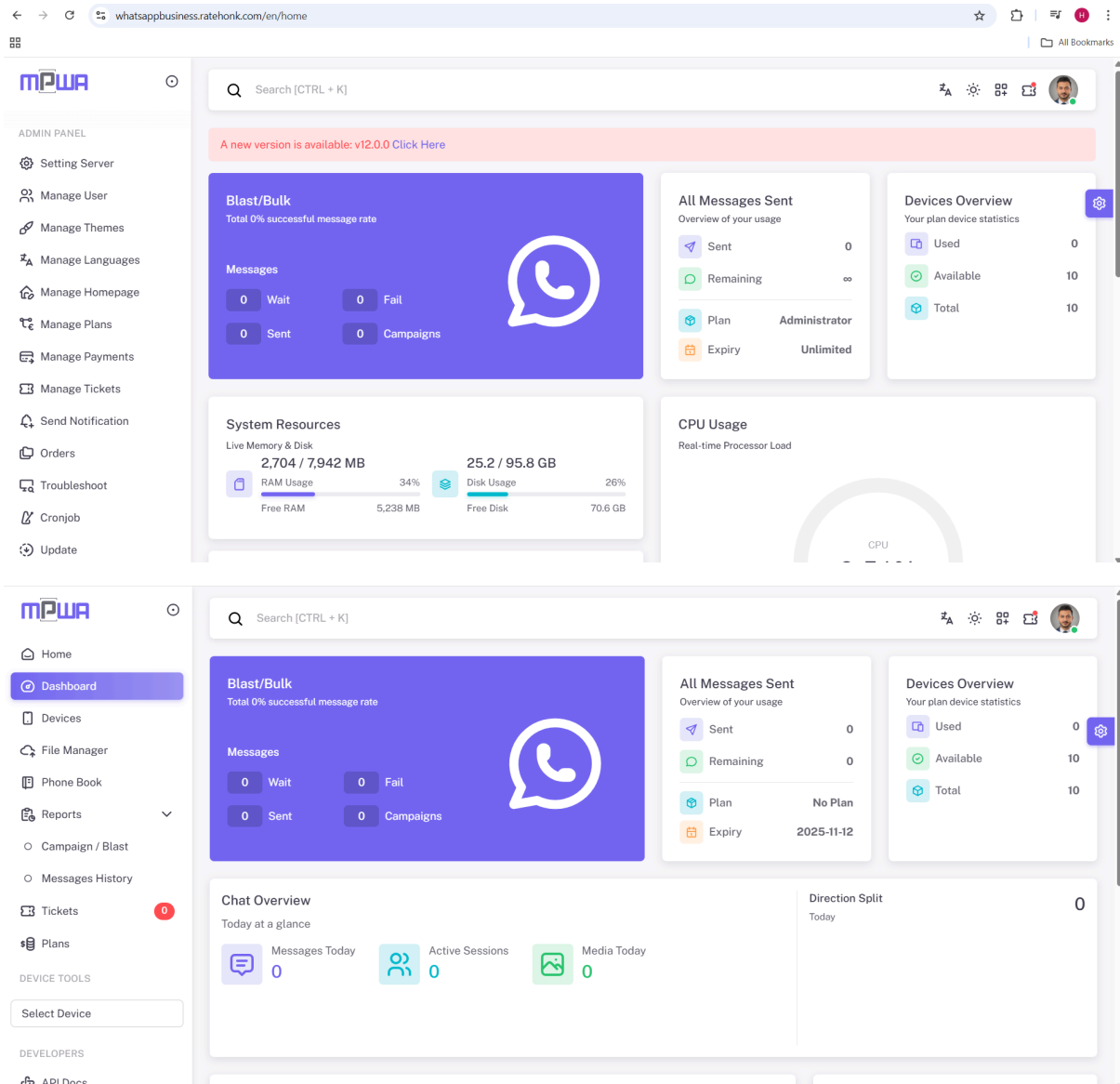
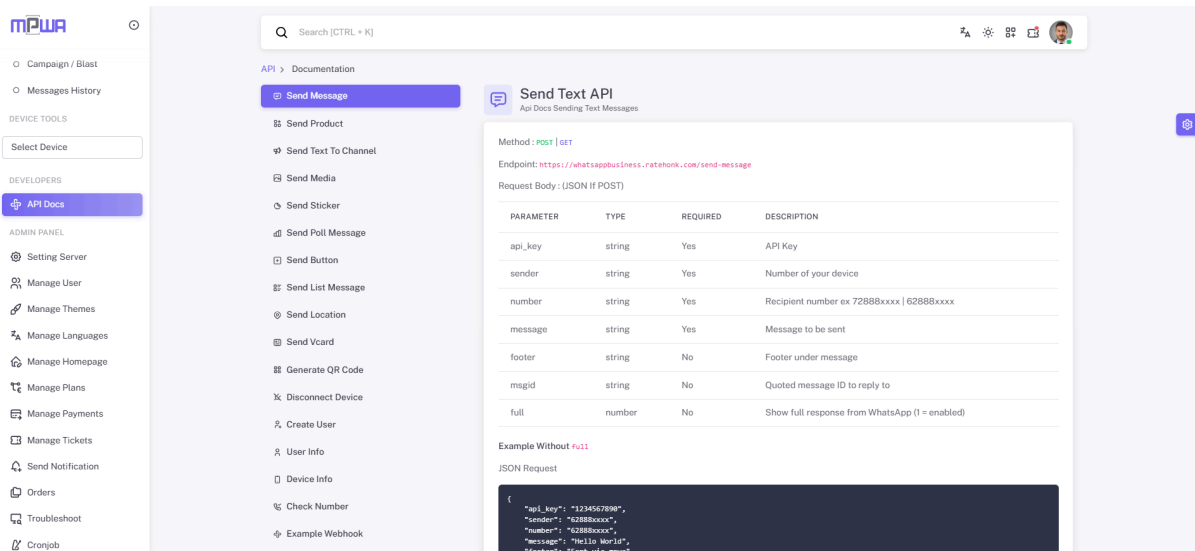


WhatsApp Web-style system

Message: How can we create a whatsapp's script in laravel where the users or admin can connect device using qr like whats web features and create apis like send-media , send-text and any other option like give in this picture first we create apis for this and integrate the apis in [react.js](https://reactjs.org/).





Usage:

- Users/Admins can connect their WhatsApp devices via QR code,
- You can send/receive messages via APIs (e.g. [send-text](#), [send-media](#), etc.),
- The APIs can be integrated with a React.js frontend,
- And everything is powered by Laravel as the backend.

Let's break it down step by step:

Step 1: Understand the Architecture

We'll create three main components:

1 Backend (Laravel)

- Handles device management, message queue, and WhatsApp session logic.
- Exposes REST APIs like [/api/send-text](#), [/api/send-media](#), [/api/devices](#), etc.
- Uses [Venom Bot](#) or [whatsapp-web.js](#) through Node.js (since WhatsApp Web reverse engineering works via Node).

💡 Laravel will communicate with a Node.js service that handles WhatsApp Web connections.

2 Node.js Service (WhatsApp Engine)

- Runs a WhatsApp client for each connected device.
- Generates the QR code (used for scanning from mobile WhatsApp).
- Sends/receives messages.
- Emits events (via WebSocket or Laravel queue) when messages arrive.

3 Frontend (React.js)

- Displays QR codes to users.
- Allows sending messages, uploading media, and viewing status.
- Calls Laravel APIs to send commands.

Step 2: Node.js WhatsApp Web Integration

Use `whatsapp-web.js` (recommended).

Install Dependencies

`npm install whatsapp-web.js express qrcode socket.io`

Example [whatsapp-server.js](#)

```
const { Client, LocalAuth } = require('whatsapp-web.js');
const qrcode = require('qrcode');
const express = require('express');
const http = require('http');
const socketIo = require('socket.io');

const app = express();
const server = http.createServer(app);
```

```

const io = socketIo(server, { cors: { origin: "*" } });

let clients = {};

app.use(express.json());

// Create WhatsApp client
app.post('/connect', (req, res) => {
  const { device_id } = req.body;

  if (clients[device_id]) {
    return res.json({ status: 'already_connected' });
  }

  const client = new Client({
    authStrategy: new LocalAuth({ clientId: device_id }),
  });

  client.on('qr', (qr) => {
    qrcode.toDataURL(qr, (err, url) => {
      io.emit('qr', { device_id, qr: url });
    });
  });

  client.on('ready', () => {
    io.emit('ready', { device_id });
    console.log(`Device ${device_id} is ready`);
  });

  client.initialize();
  clients[device_id] = client;
  res.json({ status: 'connecting' });
});

// Send text message
app.post('/send-text', async (req, res) => {
  const { device_id, number, message } = req.body;
  const client = clients[device_id];

  if (!client) return res.status(400).json({ error: 'Device not connected' });

  try {
    await client.sendMessage(number + '@c.us', message);
    res.json({ status: 'sent', number, message });
  }

```

```

    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

```

```
server.listen(3001, () => console.log('WhatsApp server running on port 3001'));
```

✅ This server:

- Creates a new WhatsApp session.
- Sends a QR code to frontend via [Socket.IO](#).
- Allows sending text messages.

Step 3: Laravel Backend Setup

Laravel acts as a middleware + database manager.

Create APIs in Laravel

Example: routes/api.php

```
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Http;
```

```
Route::post('/devices/connect', function (Request $request) {
```

```
    $response = Http::post('http://localhost:3001/connect', [
```

```
        'device_id' => $request->device_id,
```

```
    ]);
```

```
    return $response->json();
```

```
});
```

```
Route::post('/send-text', function (Request $request) {
```

```
    $response = Http::post('http://localhost:3001/send-text', [
```

```
        'device_id' => $request->device_id,
```

```
        'number' => $request->number,
```

```
        'message' => $request->message,
```

```

    });

    return $response->json();
  });

```

Step 4: React.js Frontend

Show QR Code for Device Connection

```

import React, { useState, useEffect } from "react";
import io from "socket.io-client";
import axios from "axios";

const socket = io("http://localhost:3001");

function WhatsAppConnect() {
  const [qr, setQr] = useState(null);

  const connectDevice = async () => {
    await axios.post("http://localhost:8000/api/devices/connect", {
      device_id: "device_1",
    });
  };

  useEffect(() => {
    socket.on("qr", (data) => {
      if (data.device_id === "device_1") setQr(data.qr);
    });
    socket.on("ready", () => alert("Device connected!"));
  }, []);

  return (
    <div>
      <button onClick={connectDevice}>Connect Device</button>
      {qr && <img src={qr} alt="Scan QR" />}
    </div>
  );
}

```

export default WhatsAppConnect;

✅ When you click **Connect Device**, it will:

- Hit Laravel → Laravel hits Node.js `/connect`.
- Node.js emits a QR → React listens and shows it.
- Scan QR from WhatsApp mobile → connected!

Step 5: Create More APIs (like in your screenshot)

You can extend Node.js with:

- `/send-media` → send images/files.
- `/get-chats` → list recent chats.
- `/get-messages` → fetch chat history.
- `/disconnect-device` → logout a device.
- `/send-button-message` → send template buttons.

Each of these APIs can be mirrored in Laravel (`routes/api.php`) for better control and authentication.

Step 6: Real-Time Updates (Optional)

Use **Socket.IO** or **Laravel Echo** to update React frontend in real-time:

- Show message status.
- Show incoming chats.
- Track device connection.

